Rémi PYT
Antoine RICHARD
Matthieu Sirier
Jéremy SAGNARD

Project Description

ArtConnect Application

Database 2 : Security & Automation

Step 2: Conceptual and Logical Modeling of the ArtConnect Database

Objective: Design the conceptual and logical models of the database from the identified features, applying normalization principles (up to 3NF).

Tasks:

We directly put the identifiers inside our entities as well as for our functional dependencies.

1. Identify entities and relationships

- Based on the interface and business needs, identify the different entities and their attributes.

The different entities are:
Underscored attributes correspond to arrays, they will be later implemented as a relationship to follow atomicity in the attributes.

Galleries	Exhibition	Artwork	Artist
Gallery_Id Gallery_name Gallery_address Gallery_ownerName Gallery_openingHour Gallery_contactPhone Gallery_website Gallery_rating exhibitions	Exhibition_Id Exhibition_Title Exhibition_startDate Exhibition_endDate Exhibition_description gallery Exhibition_curatorName Exhibition_theme Exhibition_artworks	Artwork_Id Artwork_Exhibition_title Artwork_creationTear Artwork_type Artwork_medium Artwork_dimensions Artwork_description Artwork_price Artwork_status artist Artwork_tags	Artist_Id Artist_name Artist_bio Artist_birthyear Artist_disciplines Artist_contactEmail Artist_phone Artist_city Artist_website Artist_socialMedia Artist_isActive artworks
Workshop	**Review**	**Booking**	**CommunityMember**
Workshop_Id Workshop_title Workshop_date Workshop_surationMinutes Workshop_maxParticipants Workshop_price instructor Workshop_location Workshop_description Workshop_level_Work	Review_Reviewer artwork Review_rating Review_comment Review_reviewDate Review_type	Workshop Member Booking_bookingDate Booking_paymentStatus	CM_Id CM_Name CM_email CM_birthYear CM_phone CM_city CM_favoriteDisciplines CM_membershipType CM_bookings CM_reviews

- Define the functional dependencies between the different attributes.

Galleries:

gallery_Id -> Gallery_name, Gallery_Address, Gallery_ownerName, Gallery_openingHour, Gallery_contactPhone, Gallery_website, Gallery_rating, exhibition

Exhibitions:

exhibition_Id, gallery_id - > Exhibition_title, Exhibition_ startDate, Exhibition_endDate, Exhibition_description, Exhibition_curatorName, Exhibition_theme, artworks

Artwork:

Artwork_id -> Artwork_title, Artwork_creationTear, Artwork_type, Artwork_medium, Artwork_dimensions, Artwork_description, Artwork_price, Artwork_status, artist, Artwork_tags

Artist:

Artist_id -> Artist_name, Artist_bio, Artist_birthyear, Artist_disciplines, Artist_contactEmail, Artist_phone, Artist_city, Artist_website, Artist_socialMedia, Artist_isActive, artworks

Workshop:

Workshop_id -> Workshop_title, Workshop_date, Workshop_durationMinutes, Workshop_maxParticipants, Workshop_price, instructor, Workshop_location, Workshop_description, Workshop_level

Review:

reviewer, artwork -> Review_rating, Review_comment, Review_reviewDate, Review_type

Booking:

Workshop_id, member_id -> Booking_bookingDate, Booking_paymentStatus

CommunityMember:

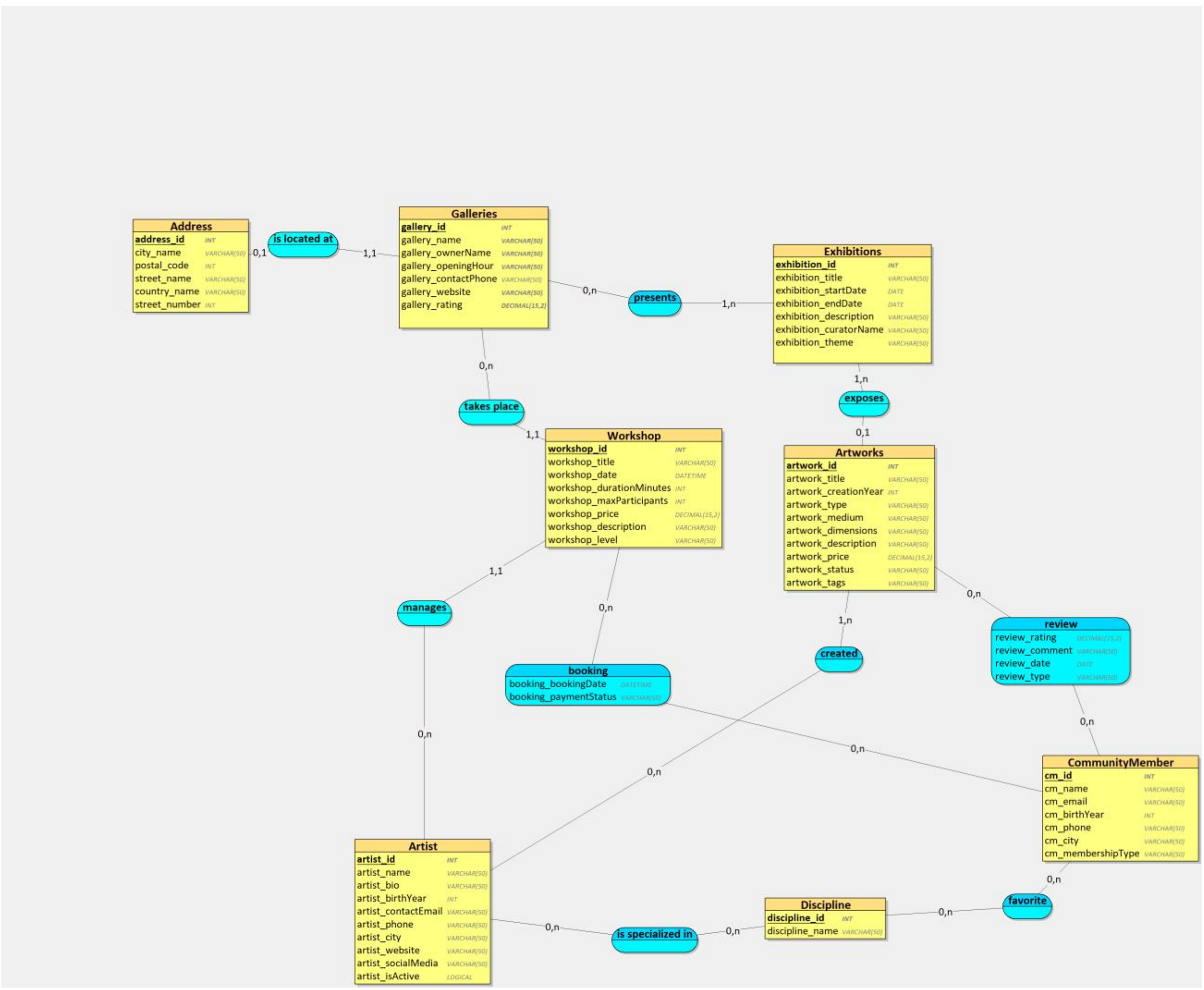
Member_id -> CM_Name, CM_email, CM_birthyear, CM_phone, CM_city, CM_favoriteDisciplines, CM_membershipType, CM_bookings, CM_reviews

- Think about the identifiers to define for each entity. You can choose one (or more) existing attribute(s) from the associated class, or define a new one.

Entity Name	ID
Galleries	Gallery_id
Exhibitions	Exhibition_id, gallery_id
Artwork	Artwork_id
Artist	Artist_id
Workshop	Workshop_id
Review	Review_id
Booking	Workshop_id, member_id
CommunityMember	Member_id

2. Create a Conceptual Data Model (CDM)

- Use an Entity-Relationship diagram (ERD) to represent entities, attributes, and relationships.



Database 2 : Security & Automation

3. Create a Logical Data Model (LDM)

-Transform the CDM into an LDM by adding primary keys, foreign keys, and data types.
- Normalize the database up to the 3rd Normal Form (3NF).

Exhibitions = (<u>exhibition_id</u> *INT*, exhibition_title *VARCHAR(50)*, exhibition_startDate *DATE*, exhibition_endDate *DATE*, exhibition_description *VARCHAR(50)*, exhibition_curatorName *VARCHAR(50)*, exhibition_theme *VARCHAR(50)***);**

Artworks = (<u>artwork_id</u> *INT*, artwork_title *VARCHAR(50)*, artwork_creationYear *INT*, artwork_type *VARCHAR(50)*, artwork_medium *VARCHAR(50)*, artwork_dimensions *VARCHAR(50)*, artwork_description *VARCHAR(50)*, artwork_price *DECIMAL(15,2)*, artwork_status *VARCHAR(50)*, artwork_tags *VARCHAR(50), #exhibition_id****);**

Artist = (<u>artist_id</u> *INT*, artist_name *VARCHAR(50)*, artist_bio *VARCHAR(50)*, artist_birthYear *INT*, artist_contactEmail *VARCHAR(50)*, artist_phone *VARCHAR(50)*, artist_city *VARCHAR(50)*, artist_website *VARCHAR(50)*, artist_socialMedia *VARCHAR(50)*, artist_isActive *LOGICAL***);**

CommunityMember = (<u>cm_id</u> *INT*, cm_name *VARCHAR(50)*, cm_email *VARCHAR(50)*, cm_birthYear *INT*, cm_phone *VARCHAR(50)*, cm_city *VARCHAR(50)*, cm_membershipType *VARCHAR(50)***);**

Discipline = (<u>discipline_id</u> *INT*, discipline_name *VARCHAR(50)***);**

Address = (<u>address_id</u> *INT*, city_name *VARCHAR(50)*, postal_code *INT*, street_name *VARCHAR(50)*, country_name *VARCHAR(50)*, street_number *INT***);**

Galleries = (<u>gallery_id</u> *INT*, gallery_name ***VARCHAR(50)***, gallery_ownerName ***VARCHAR(50)***, gallery_openingHour ***VARCHAR(50)***, gallery_contactPhone *VARCHAR(50)*, gallery_website ***VARCHAR(50)***, gallery_rating ***DECIMAL(15,2), #address_id*****);**

Workshop = (<u>workshop_id</u> *INT*, workshop_title *VARCHAR(50)*, workshop_date *DATETIME*, workshop_durationMinutes *INT*, workshop_maxParticipants *INT*, workshop_price *DECIMAL(15,2)*, workshop_description *VARCHAR(50)*, workshop_level *VARCHAR(50), #artist_id, #gallery_id***);**

created = (<u>*#artwork_id, #artist_id*</u>);

presents = (<u>*#gallery_id, #exhibition_id*</u>);

booking = (<u>*#workshop_id, #cm_id*</u>, booking_bookingDate *DATETIME*, booking_paymentStatus *VARCHAR(50)***);**

is_specialized_in = (<u>*#artist_id, #discipline_id*</u>);

favorite = (<u>*#cm_id, #discipline_id*</u>);

review = (<u>*#artwork_id, #cm_id*</u>, review_rating *DECIMAL(15,2)*, review_comment *VARCHAR(50)*, review_date *DATE*, review_type *VARCHAR(50)***);**

4. Create the database schema

- Write SQL scripts to create tables, define constraints (e.g., primary keys, foreign keys, NOT NULL), and establish relationships.

```
CREATE DATABASE ARTPROJECT;
USE ARTPROJECT;
CREATE TABLE Exhibitions(
   exhibition_id INT auto_increment,
   exhibition_title VARCHAR(50),
   exhibition_startDate DATE,
   exhibition_endDate DATE,
   exhibition_description VARCHAR(50),
   exhibition_curatorName VARCHAR(50),
   exhibition_theme VARCHAR(50),
   PRIMARY KEY(exhibition_id)
);

CREATE TABLE Artworks(
   artwork_id INT auto_increment,
   artwork_title VARCHAR(50),
   artwork_creationYear INT,
   artwork_type VARCHAR(50),
   artwork_medium VARCHAR(50),
   artwork_dimensions VARCHAR(50),
   artwork_description VARCHAR(50),
   artwork_price DECIMAL(15,2),
   artwork_status VARCHAR(50),
   artwork_tags VARCHAR(50),
   exhibition_id INT,
   PRIMARY KEY(artwork_id),
   FOREIGN KEY(exhibition_id) REFERENCES Exhibitions(exhibition_id) ON
DELETE CASCADE
);

CREATE TABLE Artist(
   artist_id INT auto_increment,
   artist_name VARCHAR(50),
   artist_bio VARCHAR(50),
   artist_birthYear INT,
   artist_contactEmail VARCHAR(50),
   artist_phone VARCHAR(50),
   artist_city VARCHAR(50),
   artist_website VARCHAR(50),
   artist_socialMedia VARCHAR(50),
   artist_isActive BOOLEAN,
   PRIMARY KEY(artist_id)
);

CREATE TABLE CommunityMember(
   cm_id INT auto_increment,
   cm_name VARCHAR(50),
   cm_email VARCHAR(50),
   cm_birthYear INT,
```

```sql
   cm_phone VARCHAR(50),
   cm_city VARCHAR(50),
   cm_membershipType VARCHAR(50),
   PRIMARY KEY(cm_id)
);

CREATE TABLE Discipline(
   discipline_id INT auto_increment,
   discipline_name VARCHAR(50),
   PRIMARY KEY(discipline_id)
);

CREATE TABLE Address(
   address_id INT auto_increment,
   city_name VARCHAR(50),
   postal_code INT,
   street_name VARCHAR(50),
   country_name VARCHAR(50),
   street_number INT,
   PRIMARY KEY(address_id)
);

CREATE TABLE Galleries(
   gallery_id INT auto_increment,
   gallery_name VARCHAR(50) NOT NULL,
   gallery_ownerName VARCHAR(50) NOT NULL,
   gallery_openingHour VARCHAR(50) NOT NULL,
   gallery_contactPhone VARCHAR(50),
   gallery_website VARCHAR(50) NOT NULL,
   gallery_rating DECIMAL(15,2) NOT NULL,
   address_id int NOT NULL,
   PRIMARY KEY(gallery_id),
   UNIQUE(address_id),
   FOREIGN KEY(address_id) REFERENCES Address(address_id) ON DELETE
CASCADE
);

CREATE TABLE Workshop(
   workshop_id INT auto_increment,
   workshop_title VARCHAR(50),
   workshop_date DATETIME,
   workshop_durationMinutes INT,
   workshop_maxParticipants INT,
   workshop_price DECIMAL(15,2),
   workshop_description VARCHAR(50),
   workshop_level VARCHAR(50),
   artist_id INT NOT NULL,
   gallery_id INT NOT NULL,
   PRIMARY KEY(workshop_id),
   FOREIGN KEY(artist_id) REFERENCES Artist(artist_id) ON DELETE CASCADE,
   FOREIGN KEY(gallery_id) REFERENCES Galleries(gallery_id) ON DELETE
CASCADE
);

CREATE TABLE created(
   artwork_id INT,
```

```sql
    artist_id INT,
    PRIMARY KEY(artwork_id, artist_id),
    FOREIGN KEY(artwork_id) REFERENCES Artworks(artwork_id) ON DELETE
CASCADE,
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id) ON DELETE CASCADE
);

CREATE TABLE presents(
    gallery_id INT,
    exhibition_id INT,
    PRIMARY KEY(gallery_id, exhibition_id),
    FOREIGN KEY(gallery_id) REFERENCES Galleries(gallery_id) ON DELETE
CASCADE,
    FOREIGN KEY(exhibition_id) REFERENCES Exhibitions(exhibition_id) ON
DELETE CASCADE
);

CREATE TABLE booking(
    workshop_id INT,
    cm_id INT,
    booking_bookingDate DATETIME,
    booking_paymentStatus VARCHAR(50),
    PRIMARY KEY(workshop_id, cm_id),
    FOREIGN KEY(workshop_id) REFERENCES Workshop(workshop_id) ON DELETE
CASCADE,
    FOREIGN KEY(cm_id) REFERENCES CommunityMember(cm_id) ON DELETE CASCADE
);

CREATE TABLE is_specialized_in(
    artist_id INT,
    discipline_id INT,
    PRIMARY KEY(artist_id, discipline_id),
    FOREIGN KEY(artist_id) REFERENCES Artist(artist_id) ON DELETE CASCADE,
    FOREIGN KEY(discipline_id) REFERENCES Discipline(discipline_id) ON
DELETE CASCADE
);

CREATE TABLE favorite(
    cm_id INT,
    discipline_id INT,
    PRIMARY KEY(cm_id, discipline_id),
    FOREIGN KEY(cm_id) REFERENCES CommunityMember(cm_id) ON DELETE
CASCADE,
    FOREIGN KEY(discipline_id) REFERENCES Discipline(discipline_id) ON
DELETE CASCADE
);

CREATE TABLE review(
    artwork_id INT,
    cm_id INT,
    review_rating DECIMAL(15,2),
    review_comment VARCHAR(50),
    review_date DATE,
    review_type VARCHAR(50),
    PRIMARY KEY(artwork_id, cm_id),
```

```sql
    FOREIGN KEY(artwork_id) REFERENCES Artworks(artwork_id) ON DELETE
CASCADE,
    FOREIGN KEY(cm_id) REFERENCES CommunityMember(cm_id) ON DELETE CASCADE
);
```